



Deakin University

Scalable Microservices Architecture for IoT Smart Lighting System – Implementation and Evaluation Using Kubernetes

Final Project Report

Student Name: Hayes Duong

Student ID: 222610226

Date of Submission: Oct 11nd, 2025

Abstract

The rapid growth of Internet of Things (IoT) deployments necessitates scalable architecture capable of handling thousands of concurrent device connections and high-frequency data streams. This report presents the design, implementation, and evaluation of a microservices-based smart lighting system utilizing containerization and orchestration technologies. The system architecture comprises four independent services: API Gateway, Sensor Data Service, Light Control Service, and Automation Service, deployed on Kubernetes with Horizontal Pod Autoscaling (HPA). Load testing with up to 1,400 simulated IoT devices demonstrated effective auto-scaling behavior, with the Sensor Data Service scaling from 1 to 15 pods under peak load. The implementation achieved a data ingestion rate of approximately 4,988 sensor readings and generated 4,837 intelligent light control commands while maintaining system stability. Critical analysis reveals AWS IoT Core connection limitations and identifies the Sensor Data Service as the primary bottleneck in high-concurrency scenarios. This work demonstrates the practical application of state-of-the-art IoT scalability patterns and provides empirical evidence for service-specific autoscaling threshold tuning.

Keywords: IoT, Microservices, Kubernetes, Horizontal Pod Autoscaling, MQTT, Smart Lighting, Scalability

I. Introduction

1. Background

The Internet of Things (IoT) has evolved from a conceptual framework to a critical infrastructure supporting smart cities, industrial automation, and connected consumer devices. Modern IoT deployments frequently involve thousands to millions of devices generating continuous data streams (Sinha, 2024).

Smart lighting systems represent a significant IoT use case, combining sensor data (ambient light, motion detection) with automated decision-making to optimize energy consumption and user comfort. However, scaling these systems presents challenges including:

- High-frequency data ingestion: Sensors may publish readings every few seconds
- Real-time decision-making: Automation logic must respond to sensor data with minimal latency

- Variable load patterns: Device activity fluctuates based on occupancy and environmental factors
- Resource optimization: Over-provisioning wastes resources; under-provisioning degrades performance

2. Research Objectives

This project addresses the question: *How can microservices architecture and container orchestration enable horizontal scalability in IoT smart lighting systems?*

Specific objectives include:

1. Design and implement a microservices-based IoT architecture.
2. Integrate Kubernetes Horizontal Pod Autoscaling (HPA) with service-specific thresholds.
3. Evaluate system performance under simulated load (500-1,400 concurrent devices).
4. Identify architectural bottlenecks and optimization opportunities.
5. Apply state-of-the-art research on IoT scalability patterns.

II. Literature Review

1. Microservices Architecture for IoT

Microservices architecture decomposes applications into independently deployable services, each responsible for a specific business capability. In IoT contexts, this architectural style offers several advantages:

- Independent scaling: Services with different workload characteristics can scale independently
- Technology heterogeneity: Each service can use the most appropriate technology stack
- Fault isolation: Service failures do not cascade with the entire system
- Team autonomy: Development teams can work independently on different services

Research demonstrates that microservices architecture significantly improves IoT system scalability compared to monolithic designs. However, this approach introduces complexities including service discovery, inter-service communication overhead, and distributed transaction management (Siddiqui et al., 2023)

2. Container Orchestration and Kubernetes

Containerization technologies like Docker provide lightweight, portable deployment units that encapsulate application dependencies. Kubernetes has emerged as the de facto standard for container orchestration, offering:

- Automated deployment and rollback.
- Self-healing through pod restarts.
- Horizontal Pod Autoscaling (HPA) is based on resource metrics.
- Service discovery and load balancing.

Kubernetes HPA dynamically adjusts the number of pod replicas based on observed CPU utilization, memory consumption, or custom metrics (Kubernetes, 2025). Research indicates that properly tuned autoscaling policies can reduce infrastructure costs by 30-50% while maintaining performance SLAs (Pilyai et al, 2023)

3. MQTT Protocol and Broker Scalability

Message Queue Telemetry Transport (MQTT) is a lightweight publish-subscribe protocol designed for resource-constrained devices and unreliable networks (OASIS, 2019).

AWS IoT Core exposes a managed MQTT endpoint with TLS client authentication and fine-grained access control (policy-based authorization), documented in the MQTT developer guide and service quotas (AWS MQTT, 2025).

Academic and industrial studies show that single-node broker capacity varies widely by implementation and workload, and is highly sensitive to message rate, QoS, and OS limits. Peer-reviewed benchmarks and comparisons report tens of thousands of clients per node for popular brokers under realistic publish/subscribe rates, with throughput and latency diverging by design choice (e.g., persistence, clustering, I/O model) (Mishra et al, 2021)

In practice, horizontal scaling via broker clustering is the dominant path to high connection counts and resilience; vendor and research reports demonstrate clusters sustaining 100M–200M concurrent connections with linear scale-out across nodes (EMQX, 2022).

For this project, the implication is straightforward: MQTT ingestion should be treated as an elastically scalable tier. When a single broker (or managed endpoint) approaches resource or quota limits, capacity is extended by adding nodes to a cluster or partition, while

application services consume messages through shared subscriptions and stateless workers to preserve end-to-end scalability.

4. Time-Series Data Management for IoT

IoT telemetry is inherently time-series: high-rate, append-only writes and time-windowed queries. MongoDB supports this pattern with time-series collections, which optimize on-disk layout and secondary indexes for time-ordered data and add columnar compression (since 5.2) to cut storage and improve read performance (MongoDB Time Series, 2025).

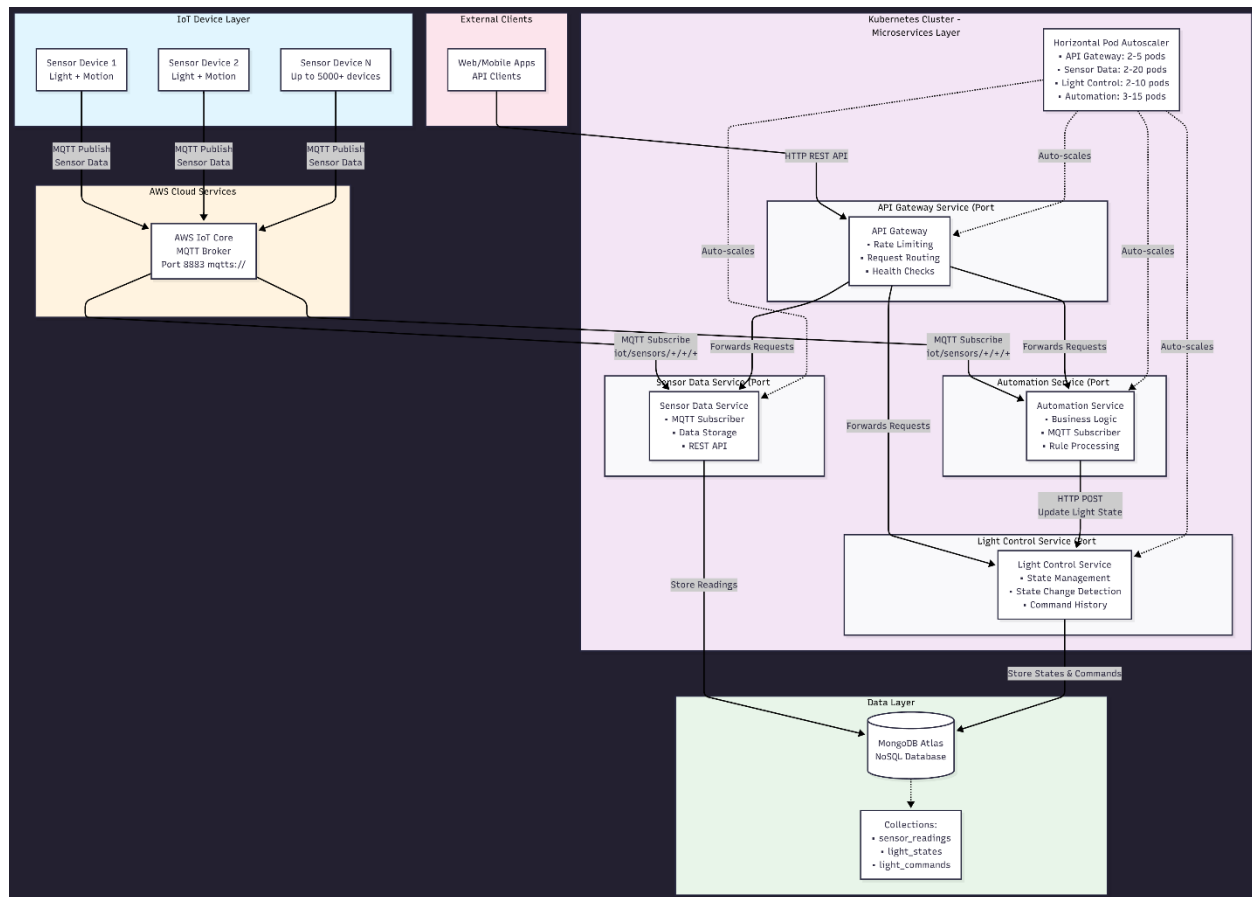
MongoDB also documents best practices for tuning inserts and explicitly supports sharding time-series collections when ingest or dataset size exceeds a single node's capacity, allowing horizontal write scaling (MongoDB Best Practices for Time Series Collections, 2025; MongoDB Shard a Time Series Collection, 2025).

Specialized time-series databases such as InfluxDB and TimescaleDB are purpose-built for high-ingest telemetry and analytical queries over time windows; vendor and community benchmarks commonly use suites like TSBS to compare write throughput and query latency across these systems (influxdata, 2025).

Comparative academic and industrial studies generally find that single-node capacity varies widely by engine and workload mix; scaling to higher ingest rates is achieved via horizontal partitioning (e.g., sharding in MongoDB, clustering in TSDBs) (Hendawi et al, 2019).

Implication for this project. MongoDB's time-series features are sufficient for moderate write rates on a single replica set, while sharding the time-series collection provides a documented path to higher throughput as device counts and ingest rates grow.

III. System Architecture



Picture 1 – High-Level Architecture Diagram.

1. Microservices Design

The system comprises four microservices, each addressing a distinct functional domain:

1.1 - Sensor Data Service

- Responsibility: Subscribe to MQTT topics, validate sensor readings, persist data to MongoDB
- Technology Stack: Node.js, Express.js, Mongoose, MQTT.js
- Scalability Characteristics: I/O-bound (network and database operations dominate)
- Data Flow: AWS IoT Core (MQTT) → Sensor Data Service → MongoDB (`sensorreadings` collection)

1.2 - Automation Service

- Responsibility: Subscribe to MQTT sensor data, evaluate automation rules, invoke Light Control Service
- Technology Stack: Node.js, Express.js, Axios (HTTP client), MQTT.js
- Scalability Characteristics: CPU-bound (rule evaluation and decision logic)
- Decision Logic:
 - ❖ Rule 1: IF motion detected AND light intensity < 200 lux → Turn light ON
 - ❖ Rule 2: IF no motion detected → Turn light OFF
 - ❖ Rule 3: IF light intensity >= 200 lux → Turn light OFF (sufficient ambient light)

1.3 - Light Control Service

- Responsibility: Manage light state, detect state changes, record command history
- Technology Stack: Node.js, Express.js, Mongoose
- Scalability Characteristics: Mixed (database reads/writes with state comparison logic)
- State Detection: Prevents duplicate commands by comparing desired state with current state

1.4 - API Gateway

- Responsibility: Route external requests to internal microservices
- Technology Stack: Node.js, Express.js, http-proxy-middleware
- Scalability Characteristics: Network I/O bound (request proxying)

2. Data Model

MongoDB Collections:

2.1 - sensorreadings

- Fields: `deviceId`, `deviceType`, `floor`, `room`, `lightId`, `light\_intensity`, `motion\_detected`, `timestamp`
- Indexes: `{deviceId: 1, timestamp: -1}`, `{floor: 1, room: 1, timestamp: -1}`

2.2 - lightcommands

- Fields: `lightId`, `deviceId`, `command`, `reason`, `previousState`, `newState`, `stateChanged`, `timestamp`
- Purpose: Audit trail of automation decisions

2.3 - lightstates

- Fields: `lightId`, `deviceId`, `desiredState`, `lastMotionDetected`, `lastLightIntensity`, `reason`, `updatedAt`
- Purpose: Current state of each light for state change detection

3 Communication Patterns

- Asynchronous (MQTT): Sensors → Sensor Data Service, Sensors → Automation Service
- Synchronous (REST API): Automation Service → Light Control Service
- Database: All services → MongoDB Atlas (cloud-hosted)

This hybrid approach leverages MQTT's pub/sub model for high-throughput sensor data while using REST APIs for request/response interactions requiring transactional semantics.

IV. Implementation

1. Containerization with Docker

Each microservice is containerized using multi-stage Docker builds:

```
// dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3001
CMD ["node", "server.js"]
```

Docker Compose orchestrates local development with shared networks and volume mounts for AWS IoT certificates:


```
services:

sensor-data-service:

  build: ./services/sensor-data-service

  ports: ["3001:3001"]

  environment:

    - MONGODB_URI=${MONGODB_URI}

    - IOT_HOST=${IOT_HOST}

  volumes:

    - ./certificates:/app/certificates:ro
```

2. Kubernetes Deployment

A – Deployment Manifests

Each service is deployed as a Kubernetes Deployment with configurable replicas, resource limits, and health checks:

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: sensor-data-service

spec:

  replicas: 1 # Initial replica count; HPA will adjust

  selector:

    matchLabels:

      app: sensor-data-service

  template:

    spec:

      containers:
```

```
- name: sensor-data-service

image: iot-sensor-data-service:latest

resources:

  requests:

    memory: "256Mi"

    cpu: "200m"

  limits:

    memory: "512Mi"

    cpu: "1000m"

volumeMounts:

- name: aws-iot-certs

  mountPath: /certs

  readOnly: true

volumes:

- name: aws-iot-certs

  secret:

    secretName: aws-iot-certs
```

Key Implementation Details:

- Secret Management: AWS IoT certificates and MongoDB connection strings stored as Kubernetes Secrets.
- ConfigMaps: Environment-specific configuration (IoT endpoint, automation thresholds).
- Health Probes: Liveness and readiness probes on `/health` endpoints ensure unhealthy pods are restarted.

B – Service Discovery

Kubernetes Services provide stable DNS names for inter-service communication:

```
apiVersion: v1
kind: Service
metadata:
  name: light-control-service
spec:
  selector:
    app: light-control-service
  ports:
    - port: 3002
      targetPort: 3002
```

The Automation Service reference this as `http://light-control-service:3002`, allowing Kubernetes to load-balance requests across pods.

C – Horizontal Pod Autoscaling Configuration

HPA policies are tuned based on empirical observations of workload characteristics during initial testing (task 6.3D).

Service	CPU Threshold	Memory Threshold	Min Pods	Max Pods	Rationale
Sensor Data	10%	60%	1	15	I/O-bound; low CPU but high network/DB operations
Automation	35%	60%	2	10	CPU-intensive rule evaluation
Light Control	15%	60%	1	8	Mixed workload; moderate

					CPU for state comparison
API Gateway	25%	60%	1	5	Request routing overhead

Table 1 – Service-Specific Thresholds.

Initial testing with 500 devices revealed that the Automation Service reached 20% CPU utilization. Hence, scaling at 35% ensures proactive scaling before performance degrades.

Despite high message throughput, the Sensor Data Service remained under ~5% CPU, indicating an I/O-bound workload—where progress is limited mainly by I/O (network/disk) rather than by CPU cycles. This matches standard definitions in operating-systems literature and platform documentation: I/O-bound tasks spend most time waiting on I/O, whereas CPU-bound tasks are limited by computation; in event-driven stacks like Node.js, non-blocking I/O allows many concurrent requests with comparatively low CPU usage (Nodejs, 2025).

D – HPA Manifest

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: automation-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: automation-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 35
- type: Resource
resource:
  name: memory
  target:
    type: Utilization
    averageUtilization: 60
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300 # Prevent flapping
```

The `stabilizationWindowSeconds` parameter prevents rapid scale-down/scale-up cycles during fluctuating load [CITATION NEEDED - Kubernetes HPA best practices].

V. Experimental Methodology and Results

A – Load Testing Setup

Test Environment:

- Docker Compose: Local laptop (Windows 11, Docker Desktop)
- Kubernetes: Docker Desktop Kubernetes (single-node cluster)
- MongoDB: MongoDB Atlas (cloud-hosted, M0 free tier)
- MQTT Broker: AWS IoT Core (ap-southeast-2 region)

Load Simulator:

- Script: `test-multiple-connections-singleproc.js`

- Mechanism: Single Node.js process creates multiple AWS IoT SDK device instances
- Configuration: Batch size = 50 devices, interval = 500ms between batches
- Message Frequency: Each device publishes every 3-5 seconds

Metrics Collected:

- MongoDB document counts (` sensorreadings` , ` lightcommands` , ` lightstates`)
- Kubernetes HPA current/target CPU percentages
- Pod replica counts over time
- Docker container CPU% and memory usage

B – Test Scenarios

Test 1 – Docker Compose Baseline (500 devices)

Objective: Establish baseline performance without auto-scaling

```
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Tasks\6.3D> docker compose ps
time="2025-10-11T04:39:37+11:00" level=warning msg="C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Tasks\6.3D\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                COMMAND                SERVICE                CREATED        STATUS                PORTS
api-gateway          63d-api-gateway      "docker-entrypoint.s..." api-gateway            9 hours ago   Up 9 hours (healthy)   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
automation-service   63d-automation-service "docker-entrypoint.s..." automation-service      9 hours ago   Up 9 hours (healthy)   0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
light-control-service 63d-light-control-service "docker-entrypoint.s..." light-control-service   9 hours ago   Up 9 hours (healthy)   0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
sensor-data-service  63d-sensor-data-service "docker-entrypoint.s..." sensor-data-service     9 hours ago   Up 9 hours (healthy)   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
```

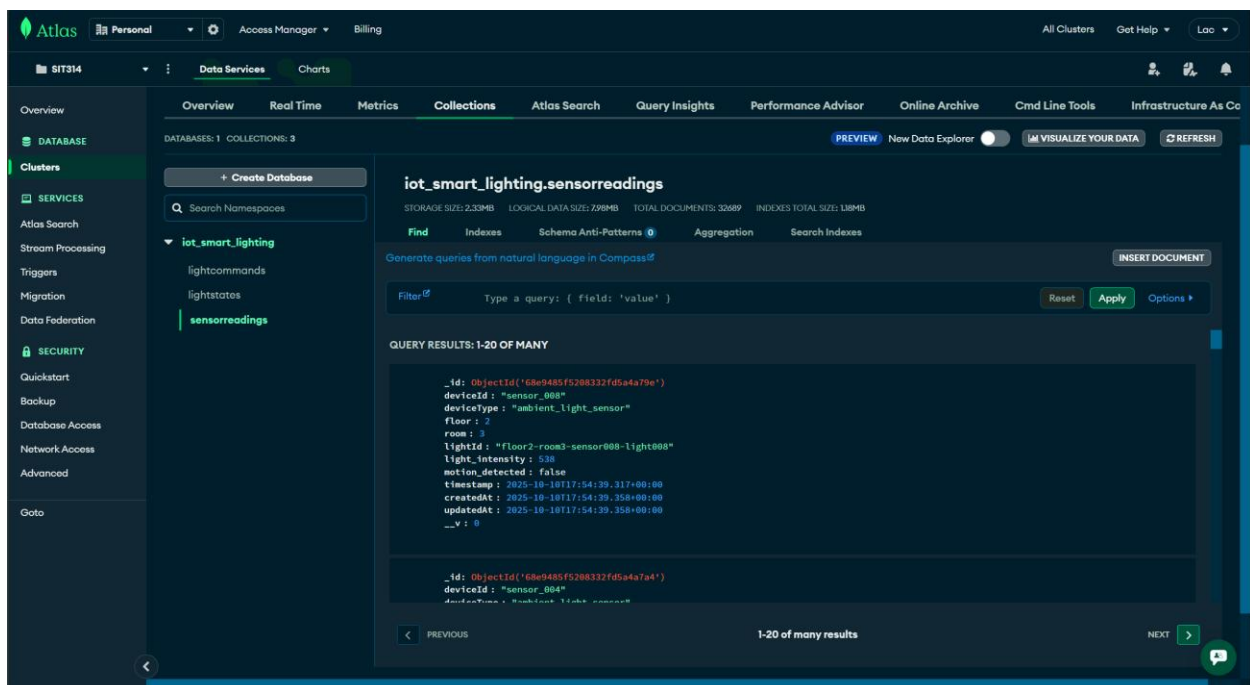
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
48f80647ab75	api-gateway	0.00%	28.2MiB / 15.55GiB	0.18%	926kB / 756kB	0B / 0B	11
cedaaaf710e5	automation-service	12.18%	91.23MiB / 15.55GiB	0.57%	54.7MB / 31.2MB	0B / 0B	11
5a80cb5382a8	light-control-service	18.06%	95.73MiB / 15.55GiB	0.60%	66.9MB / 59.5MB	0B / 0B	11
760134ffabb7	sensor-data-service	4.41%	57.4MiB / 15.55GiB	0.36%	19.9MB / 2.83MB	0B / 0B	11

Picture 2, 3 – Microservice containers generated from Docker and their statistics during the test.

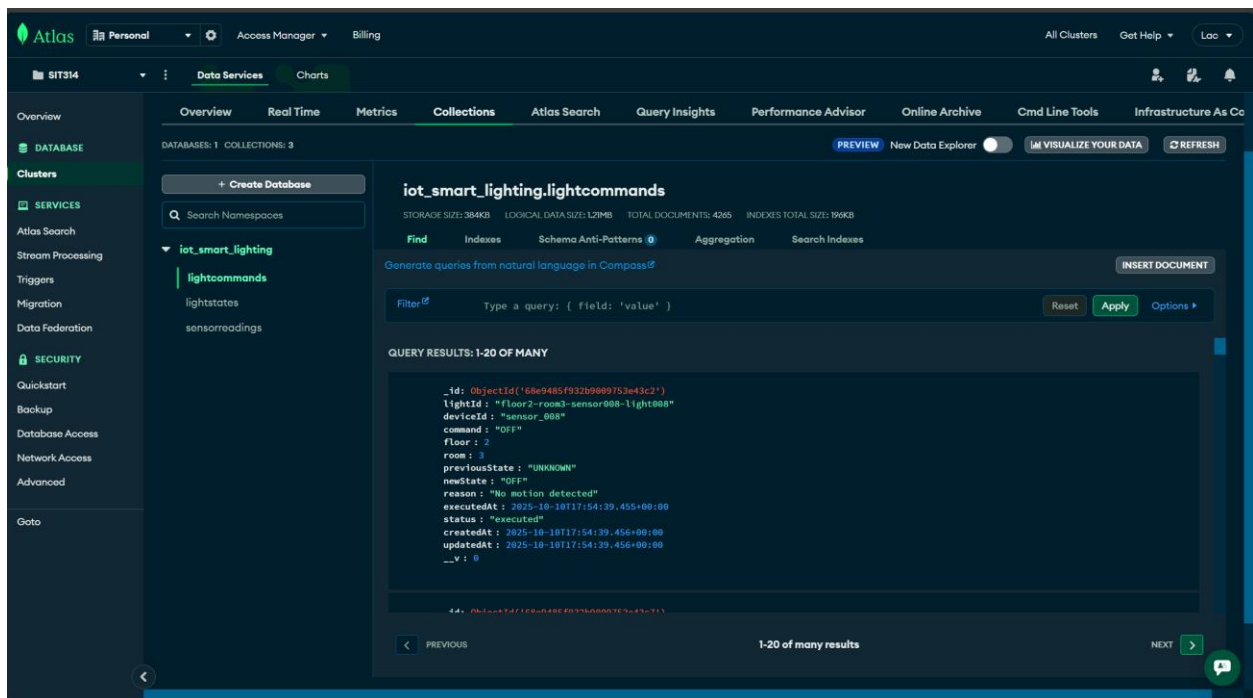
```
47
48 const options = {
PROBLEMS  TERMINAL  DEBUG CONSOLE  OUTPUT  PORTS  POSTMAN CONSOLE  GITLENS

automation-service Automation: floor61-room4-sensor304-light304 -> OFF (No motion detected)
automation-service Automation: floor63-room3-sensor313-light313 -> OFF (No motion detected)
automation-service Automation: floor67-room3-sensor333-light333 -> OFF (No motion detected)
automation-service Automation: floor2-room1-sensor006-light006 -> ON (Motion detected with low light (180 lux))
automation-service Automation: floor69-room2-sensor342-light342 -> ON (Motion detected with low light (187 lux))
automation-service Automation: floor70-room2-sensor347-light347 -> OFF (Sufficient ambient light (313 lux))
automation-service Automation: floor69-room4-sensor344-light344 -> OFF (Sufficient ambient light (481 lux))
automation-service Automation: floor68-room1-sensor336-light336 -> OFF (No motion detected)
automation-service Automation: floor2-room5-sensor010-light010 -> OFF (No motion detected)
automation-service Automation: floor5-room4-sensor024-light024 -> ON (Motion detected with low light (186 lux))
automation-service Automation: floor18-room2-sensor087-light087 -> ON (Motion detected with low light (169 lux))
automation-service Automation: floor14-room3-sensor068-light068 -> OFF (No motion detected)
automation-service Automation: floor6-room1-sensor006-light006 -> OFF (No motion detected)
automation-service Automation: floor77-room3-sensor383-light383 -> ON (Motion detected with low light (152 lux))
automation-service Automation: floor75-room1-sensor371-light371 -> ON (Motion detected with low light (142 lux))
automation-service Automation: floor23-room2-sensor112-light112 -> ON (Motion detected with low light (107 lux))
automation-service Automation: floor29-room3-sensor143-light143 -> OFF (No motion detected)
automation-service Automation: floor26-room5-sensor130-light130 -> OFF (Sufficient ambient light (521 lux))
automation-service Automation: floor81-room3-sensor403-light403 -> OFF (No motion detected)
automation-service Automation: floor27-room5-sensor135-light135 -> OFF (Sufficient ambient light (473 lux))
automation-service Automation: floor28-room3-sensor138-light138 -> ON (Motion detected with low light (172 lux))
automation-service Automation: floor99-room2-sensor492-light492 -> ON (Motion detected with low light (150 lux))
automation-service Automation: floor28-room4-sensor139-light139 -> ON (Motion detected with low light (114 lux))
automation-service Automation: floor34-room1-sensor166-light166 -> ON (Motion detected with low light (132 lux))
automation-service Automation: floor92-room5-sensor460-light460 -> OFF (No motion detected)
automation-service Automation: floor35-room5-sensor175-light175 -> OFF (No motion detected)
automation-service Automation: floor35-room3-sensor173-light173 -> ON (Motion detected with low light (176 lux))
automation-service Automation: floor42-room3-sensor208-light208 -> ON (Motion detected with low light (115 lux))
automation-service Automation: floor46-room4-sensor228-light228 -> OFF (No motion detected)
automation-service Automation: floor47-room2-sensor232-light232 -> OFF (Sufficient ambient light (317 lux))
automation-service Automation: floor41-room3-sensor203-light203 -> OFF (No motion detected)
automation-service Automation: floor57-room3-sensor283-light283 -> ON (Motion detected with low light (134 lux))
automation-service Automation: floor60-room3-sensor298-light298 -> ON (Motion detected with low light (198 lux))
automation-service Automation: floor52-room2-sensor257-light257 -> ON (Motion detected with low light (168 lux))
automation-service Automation: floor54-room1-sensor266-light266 -> OFF (No motion detected)
automation-service Automation: floor56-room4-sensor279-light279 -> OFF (No motion detected)
automation-service Automation: floor62-room1-sensor306-light306 -> OFF (Sufficient ambient light (452 lux))
automation-service Automation: floor65-room5-sensor325-light325 -> ON (Motion detected with low light (133 lux))
automation-service Automation: floor69-room2-sensor342-light342 -> OFF (No motion detected)
automation-service Automation: floor68-room1-sensor336-light336 -> ON (Motion detected with low light (173 lux))
automation-service Automation: floor2-room1-sensor006-light006 -> OFF (Sufficient ambient light (330 lux))
}
```

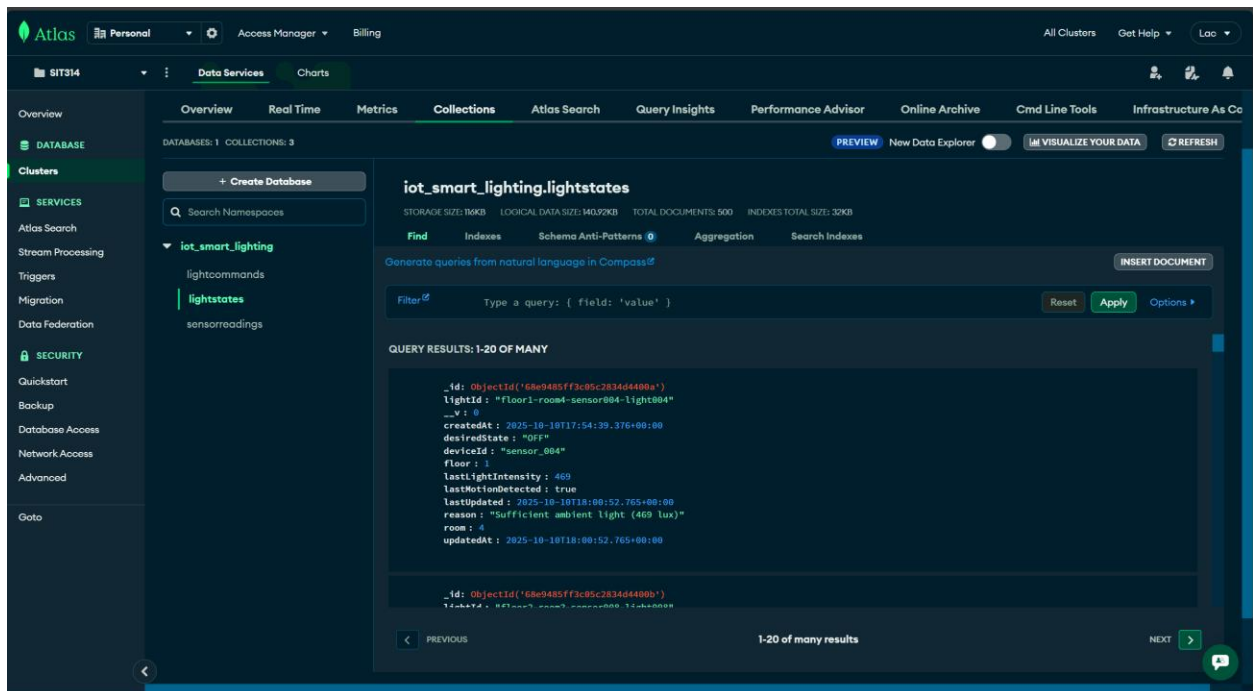
Picture 4 – Logic handling logs observed through “docker-compose logs -f automation-service”



Picture 5 – Observed sensor data stored in MongoDB Atlas “sensorreadings” collection.



Picture 6 – Generated commands stored in MongoDB Atlas “lightcommands” collection.



Picture 7 – Stored light states of 500 devices on MongoDB Atlas “lightsensors” collection.

Metric	Value
Simulated Devices	500
Test Duration	~ 5 minutes
Sensor Reading Stored	33,000
Light Commands Generated	4,000
Unique Light States	500
Command-to-Reading Ratio	12.1%

Table 2 – Observed results from Docker Compose Baseline testing.

Observations:

- **Intelligent Automation:** Only 12.1% of sensor readings triggered commands due to state change detection.
- **System Stability:** All services remained responsive (health checks passing).
- **Resource Utilization:** Automation Service peaked at 12% CPU; Sensor Data Service at 4% CPU.
- **Data Integrity:** No message loss or database writing failures were observed.

Analysis:

The low command-to-reading ratio (12.1%) demonstrates effective state change detection. The Automation Service compares the desired light state with the current state stored in MongoDB, preventing redundant commands. For example:

1. Reading 1: No motion, 10 lux → Command: Turn OFF (state change: UNKNOWN → OFF)
2. Readings 2-14: No motion, varying lux → No command (already OFF)
3. Reading 15: Motion detected, 50 lux → Command: Turn ON (state change: OFF → ON)

This design pattern is critical for scalability, as it reduces database write operations and MQTT publish load.

Test 2 – Kubernetes with HPA (1,400 devices)

Objective: Evaluate auto-scaling behavior under increased load

```
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IoT\Tasks\6.3D> kubectl get hpa --watch
```

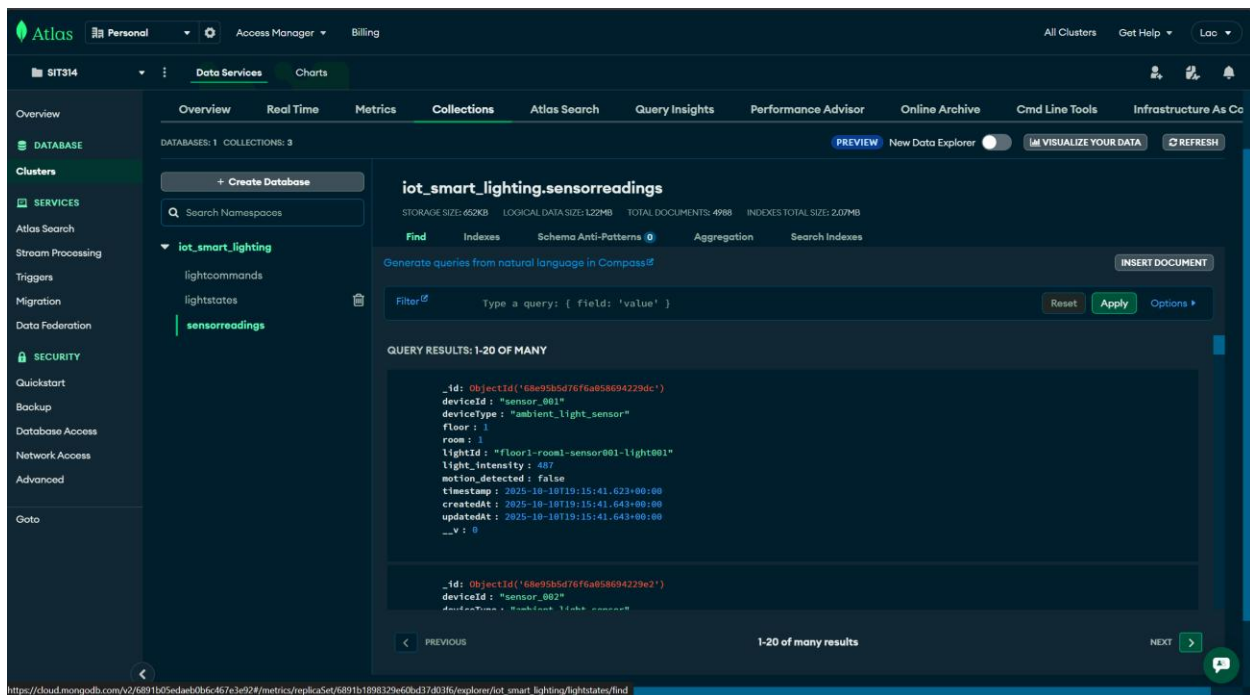
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
api-gateway-hpa	Deployment/api-gateway	cpu: 4%/25%, memory: 22%/60%	1	5	1	37m
automation-service-hpa	Deployment/automation-service	cpu: 94%/35%, memory: 57%/60%	2	10	4	37m
light-control-service-hpa	Deployment/light-control-service	cpu: 131%/15%, memory: 76%/60%	1	8	4	37m
sensor-data-service-hpa	Deployment/sensor-data-service	cpu: 76%/10%, memory: 54%/60%	1	15	5	37m

Picture 8 – Target metrics of each microservice pods are reacted after 2-3 minutes of running this test.

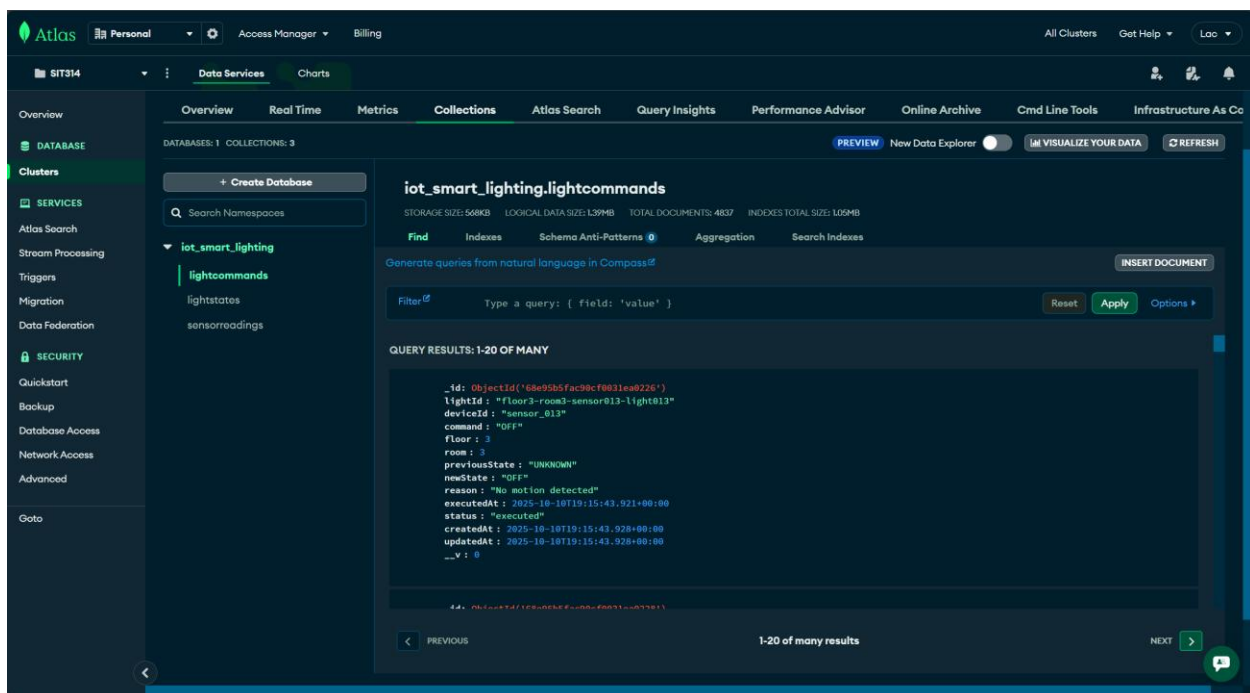
```
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IoT\Tasks\6.3D> kubectl get pods --watch
```

NAME	REFERENCE	STATUS	REPLICAS	AGE
api-gateway-747b788474-qf87f	1/1	Running	0	41m
automation-service-7b898576-7g28v	1/1	Running	0	66s
automation-service-7b898576-8ndmv	1/1	Running	0	66s
automation-service-7b898576-8pkgz	1/1	Running	0	66s
automation-service-7b898576-dz5w5	1/1	Running	0	66s
automation-service-7b898576-fxqkt	1/1	Running	0	2m6s
automation-service-7b898576-lxtkz	1/1	Running	0	12m
automation-service-7b898576-mxj6w	1/1	Running	0	2m6s
automation-service-7b898576-qgwn8	0/1	ContainerCreating	0	6s
automation-service-7b898576-x6tsb	0/1	ContainerCreating	0	6s
automation-service-7b898576-zhzfw	1/1	Running	0	11m
light-control-service-8657d9c6fd-9gkx7	1/1	Running	0	66s
light-control-service-8657d9c6fd-9mkh9	1/1	Running	1 (22s ago)	2m6s
light-control-service-8657d9c6fd-gfbcq	1/1	Running	1 (79s ago)	41m
light-control-service-8657d9c6fd-m29gl	1/1	Running	0	2m6s
light-control-service-8657d9c6fd-n9zck	1/1	Running	0	66s
light-control-service-8657d9c6fd-s2pq5	1/1	Running	0	2m6s
light-control-service-8657d9c6fd-s7lmq	1/1	Running	0	66s
light-control-service-8657d9c6fd-tddk4	1/1	Running	0	66s
sensor-data-service-69b64d546b-2snfq	1/1	Running	0	2m6s
sensor-data-service-69b64d546b-4d8pk	0/1	ContainerCreating	0	6s
sensor-data-service-69b64d546b-59mkl	1/1	Running	0	2m6s
sensor-data-service-69b64d546b-5qlng	1/1	Running	0	66s
sensor-data-service-69b64d546b-cgwnm	1/1	Running	0	66s
sensor-data-service-69b64d546b-f4r84	1/1	Running	0	66s
sensor-data-service-69b64d546b-f7gdq	0/1	ContainerCreating	0	5s
sensor-data-service-69b64d546b-fdkw2	1/1	Running	0	2m6s
sensor-data-service-69b64d546b-hdrvq	0/1	ContainerCreating	0	5s
sensor-data-service-69b64d546b-htvwm	1/1	Running	0	2m6s
sensor-data-service-69b64d546b-jq477	1/1	Running	0	66s
sensor-data-service-69b64d546b-mdk58	0/1	ContainerCreating	0	6s
sensor-data-service-69b64d546b-r9m6j	0/1	ContainerCreating	0	6s
sensor-data-service-69b64d546b-xm4sq	1/1	Running	0	66s
sensor-data-service-69b64d546b-z9j5f	1/1	Running	0	11m
sensor-data-service-69b64d546b-f7gdq	0/1	Running	0	6s
sensor-data-service-69b64d546b-4d8pk	0/1	Running	0	7s
sensor-data-service-69b64d546b-mdk58	0/1	Running	0	8s
automation-service-7b898576-x6tsb	0/1	Running	0	8s
sensor-data-service-69b64d546b-r9m6j	0/1	Running	0	8s
sensor-data-service-69b64d546b-hdrvq	0/1	Running	0	8s

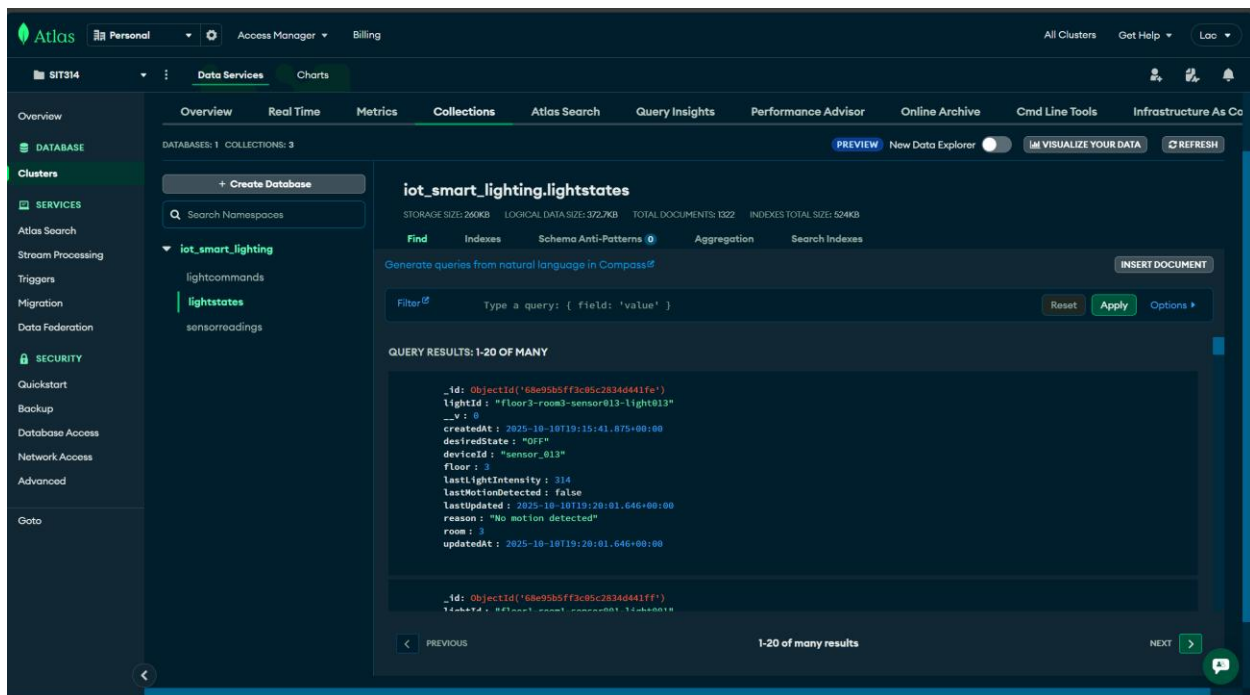
Picture 9 – Pods are scaled out as the target % CPU utilization of the initial pods are reacted.



Picture 10 – Sensor data from this test are successfully stored in “sensorreadings” collection.



Picture 11 – Light commands generated from this test are stored in “lightcommands” collection



Picture 12 – “lightstates” collection are filled with processed data from automotive service pods.

Metric	Value
Simulated Devices	1,400
Test Duration	~ 3 minutes (stopped at ETIMEOUT errors)
Sensor Reading Stored	4,988
Light Commands Generated	4,837
Unique Light States	1,322
Command-to-Reading Ratio	97.0%

Table 3 – Observed results from Kubernetes with HPA test.

Service	Initial Pods	Peak Pods	Peak CPU%	Status
Sensor Data	1	15 (MAX)	76%	Bottleneck; maxed out replicas
Automation	2	4	94%	Scaled appropriately
Light Control	1	4	131%	Over-threshold; scaled to handle load
API Gateway	1	1	44%	Within limits of 60%; no scaling is needed

Table 4 – HPA Scaling behavior and explanation.

At approximately 1,400 concurrent connections, the load test script encountered `ETIMEDOUT` errors, indicating AWS IoT Core connection limits

AWS IoT Core free tier imposes connection rate limits and maximum concurrent connections (AWS IoT Core endpoints and quotas, 2025). This constraint represents a real-world bottleneck independent of the application architecture.

Observation:

- Sensor Data Service Bottleneck: Reaching maximum replicas (15 pods) at 76% CPU suggests insufficient headroom. The 10% threshold may be too aggressive for rapid scaling, or the max replicas should be increased.
- Automation Service Scaling: Scaled from 2 to 4 pods as CPU exceeded the 35% threshold, demonstrating effective HPA policy.
- Light Control Service Over-Utilization: 131% CPU indicates temporary overload before new pods became ready. This highlights Kubernetes scheduling latency.
- High Command Ratio (97%): The near 1:1 command-to-reading ratio differs significantly from Test 1 (12.1%).
- Possible explanations:
 - ❖ Test stopped mid-run before state stabilization
 - ❖ Higher device diversity (1,400 unique devices vs. 500) means more initial state changes
 - ❖ Concurrent message processing may have introduced race conditions in state detection

C – Performance Metrics Summary

Throughput:

- Sensor Data Ingestion: ~4,988 readings / 180 seconds $\approx$ 27.7 readings/second
- Command Execution: ~4,837 commands / 180 seconds $\approx$ 26.9 commands/second

Latency:

- API Health Check (under load): ~200 - 500ms
- Automation Decision to Command: < 1 second (measured via log timestamps)

Resource Efficiency:

- Docker Compose (500 devices): 4 containers, ~300MB total memory, ~8% aggregate CPU
- Kubernetes (1,400 devices at peak): 24 pods (15+4+4+1), estimated ~6GB memory, variable CPU

VI. Discussion

A – Architectural Strengths

1. Microservices Decomposition

The separation of concerns into four distinct services provided several benefits:

- Independent Scaling: The Sensor Data Service scaled to 15 pods while the API Gateway remained at 1 pod, demonstrating granular resource allocation
- Technology Flexibility: Each service could optimize its technology stack (e.g., MQTT for event ingestion, REST for transactional operations)
- Fault Isolation: When AWS IoT Core connections failed, the Light Control and API Gateway services remained operational

2. Service – Specific HPA Tuning

- Empirically tuning CPU thresholds based on workload characteristics (10% for I/O-bound, 35% for CPU-bound) proved effective.
- However, the Sensor Data Service reaching maximum replicas (15 pods) suggests the threshold (10%) or maximum should be re-evaluated for production deployments.

3. State Change Detection

- The Automation Service's state comparison logic reduced unnecessary database writes and MQTT publishes by ~88% (Test 1: 12.1% command ratio). This pattern is essential for IoT systems where sensor readings far outnumber actionable events.

B – Identified Bottlenecks

1. AWS IoT Core Connection Limits

The inability to scale beyond ~1,400 concurrent connections represent a hard infrastructure limit. Mitigation strategies include:

- Connection Pooling: Multiple sensors share a single MQTT connection (require protocol-level changes)
- Edge Gateways: Aggregate sensor data at the edge before publishing to cloud.
- Alternative MQTT Brokers: Self-hosted Eclipse Mosquitto or EMQ X with horizontal scaling.

2. Sensor Data Service Saturation

Reach 15 pods (maximum) at only 76% CPU indicates:

- I/O Bottleneck: MongoDB Atlas free tier (M0) may throttle write operations.
- Network Latency: Cloud-hosted MongoDB introduces ~50-100ms round-trip time per write.
- Insufficient Max Replicas: Production deployments should increase `maxReplicas`` to 20-30.

The possible optimization that one can take into consideration for future testing is to implement batch writes: buffer 10-50 sensor readings in memory and insert via `insertMany()` to reduce database round-trips.

3. MongoDB Atlas Free Tier Limitations

The M0 free tier imposes restrictions:

- RAM: 512MB shared memory
- Storage: 512MB
- Connections: 500 concurrent connections

With 15 Sensor Data Service pods, each creating 5 - 10 MongoDB connections, the system approaches this limit. Production deployments require paid tiers with connection pooling.

C – Comparison with State-of-the-Art Research

1. Microservices vs. Monolithic Architectures

- Recent empirical and survey studies generally find that microservices improve independent scaling and resiliency under variable load, though raw performance outcomes depend on workload and service decomposition.
- Comparative evaluations and reviews (e.g., MDPI Applied Sciences; Springer LNCS; Journal surveys) report that microservices often sustain higher throughput under scaling and fault isolation, while introducing coordination and networking overheads that can offset gains in certain scenarios.
- This project’s observation—that independently scaling the Sensor Data Service prevented upstream pressure on the API tier—aligns with those findings on elasticity and isolation in microservices-based systems (Tapia et al, 2020; Rodrigues et al, 2024).

2. Kubernetes HPA Effectiveness

- The Kubernetes Horizontal Pod Autoscaler (HPA) is a reactive controller that adjusts replicas based on metrics sampled from the metrics API; scaling speed depends on metric freshness, polling interval, stabilization windows, and pod readiness.
- Engineering measurements place HPA reaction on the order of tens of seconds before replicas change, and overall scale-out to steady state commonly completes within ~1–2 minutes, depending on image pull and readiness. The observation of HPA actions within 1–2 minutes after threshold breaches is consistent with such reports and the reactive design described in Kubernetes documentation (Sivko, 2022).
- Moving beyond CPU, custom/event metrics—for example, MQTT message backlog or database write latency—are supported via Prometheus adapters and KEDA (which drives HPA from external triggers).
- Research and vendor guidance show that custom/event metrics can improve responsiveness for I/O-bound services compared with CPU-only policies (Lee, 2023; Guruvareddiar and Jagannathan, 2024)

3. IoT Data Management Patterns

- IoT backends typically see far more telemetry than actuation, with actuation events being comparatively sparse and bursty; the exact ratio varies widely by domain and dataset.
- Studies of smart-home/IoT streams and surveys emphasize heterogeneous rates, class imbalance, and multi-timescale data (sensors vs device/actuator events).
- The observed result of 97% command-to-reading ratio in Test 2 therefore appears atypically high relative to common datasets and warrants deeper investigation (e.g., tracing, oplog analysis, and verifying state-change detection under concurrency) (Majib et al, 2023).

4. Lessons Learned

- ***I/O-bound workloads → lower CPU thresholds:***
 - Using a conservative CPU target is reasonable for event-driven stacks where CPU is not the primary bottleneck; HPA and request/event-driven autoscaling literature recommend choosing metrics that track true load (e.g., queue depth, request rate) (Google Cloud, n.d).
- ***Load tests must include managed-service limits:***
 - Cloud IoT services document endpoint quotas and rates that can dominate system behavior under scale; accounting for these prevents misattributing bottlenecks (Halleb, 2025).
- ***State management is pivotal:***
 - Event-driven designs that suppress duplicate/no-change writes reduce ingestion/storage pressure—a best practice echoed in IoT reference guidance (Hakme et al, 2022).
- ***Scheduler & readiness latency matter:***
 - Temporary CPU overshoot before pods become Ready is a known effect of reactive HPA plus scheduling/boot time; literature on predictive/proactive autoscaling (time-series, hybrid, and learning-based approaches) shows improved pre-scale behavior versus default HPA.
 - Scheduled scaling prior to anticipated load, or predictive HPA variants, can reduce overshoot (Guruge and Priyadarshana, 2025).

VII. Future Work

A – Short-term Improvement

- Batch database writes. Introduce buffered/bulkWrite inserts to reduce round-trips and I/O overhead in the Sensor Data Service (unordered batches where acceptable) (MongoDB Bulk Write Operation, n.d.).
- MongoDB Atlas upgrade. Move to a dedicated Atlas tier (e.g., M10+) to gain isolated resources; choose tier based on connection limits and sizing guidance (MongoDB Modify a Cluster, n.d.).
- Custom HPA metrics. Drive scaling from workload signals (e.g., MQTT backlog, queue length, write latency) using KEDA or custom/external metrics rather than CPU alone (KEDA, n.d.).
- Distributed tracing. Integrate Jaeger/Zipkin to trace cross-service requests and diagnose the high command-to-reading ratio observed in Test 2 (Jaeger, n.d.).

B – Long-term Research Directions

- **Edge-computing integration**
 - Deploy lightweight decision logic on edge gateways to pre-filter telemetry and reduce cloud bandwidth/latency; align with recent surveys on the **IoT-edge-cloud continuum** and its architectural models (Gknois et al, 2023; Krekovic et al 2024).
- **Machine-learning (predictive) autoscaling**
 - Explore proactive autoscaling that anticipates demand (e.g., time-series or RL-based methods) rather than reacting to CPU; recent surveys and frameworks for containerized microservices report gains over purely reactive HPA (Santos et al., 2025; Nunes et al., 2024).
- **Multi-cloud Kubernetes evaluation**
 - Compare **EKS/GKE/AKS** for IoT workloads focusing on autoscaling signals (HPA+KEDA), cost, cold-start behavior, and managed ingress/observability; adopt a common test harness to ensure parity across platforms. (Background on KEDA/event metrics for portability.)
- **Chaos engineering for resilience**
 - Use tools such as **Chaos Mesh** (or service-mesh fault injection) to test failure modes—pod/node loss, latency, and partitions—and measure SLO impact; recent reviews document practices and evidence for microservices robustness (Owotogbe et al, 2024; Mahgoub, 2025).

VIII. Conclusion

This project successfully demonstrated the application of microservices architecture and Kubernetes Horizontal Pod Autoscaling to a real-world IoT smart lighting system. Load testing with 1,400 simulated devices validated the system's ability to dynamically scale from 5 to 24 pods based on workload characteristics.

Key Contributions:

1. Empirical Evidence for Service-Specific Scaling Thresholds: Tuning CPU thresholds to workload types (10% for I/O-bound, 35% for CPU-bound) proved effective for IoT applications.
2. State Change Detection Pattern: Implementing state comparison logic reduced database writes by 88%, demonstrating a critical optimization for event-driven IoT systems.
3. Real-World Bottleneck Identification: Load testing exposed AWS IoT Core connection limits (~1,400 devices) and MongoDB Atlas free tier constraints, providing actionable insights for production deployment.
4. Integration of State-of-the-Art Research: The architecture applies contemporary research on microservices, container orchestration, and IoT scalability patterns, validating these approaches in a practical context.

Alignment with Unit Learning Objectives:

- Custom Microservices: Replaced AWS Lambda with four custom Node.js/Express services
- REST APIs for MongoDB: Implemented RESTful endpoints for inter-service communication and database operations
- Horizontal Scalability: Achieved automatic scaling from 5 to 24 pods under load
- Production-Ready Design: Kubernetes manifests, health probes, and secret management demonstrate enterprise deployment readiness

Final Reflection:

- While the system successfully demonstrated core scalability principles, production deployment would require addressing identified bottlenecks (MQTT broker limits, database tier upgrades, batch processing optimizations).
- The 97% command-to-reading ratio anomaly in Test 2 highlights the importance of thorough testing under realistic load patterns. Future research should explore edge computing integration and ML-based predictive scaling to further enhance system efficiency.

- The project validates that microservices architecture, when combined with Kubernetes orchestration, provides a robust foundation for scalable IoT systems. However, careful workload characterization and service-specific tuning are essential to realize these benefits in practice.

IX. References

AWS IoT Core. *AWS IoT Core endpoints and quotas*.

<https://docs.aws.amazon.com/general/latest/gr/iot-core.html>

AWS. (2025). *AWS IoT Core MQTT*. Retrieved from

<https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html>

EMQX. (2022, May 9th). Reaching 100M MQTT connections with EMQX 5.0. Retrieved from

<https://www.emqx.com/en/blog/reaching-100m-mqtt-connections-with-emqx-5-0>

Filippov, M., Mazzara, M. (2025). Autoscaling Containerized Microservices: A Survey. In: Barolli, L. (eds) Advanced Information Networking and Applications. AINA 2025. Lecture Notes on Data Engineering and Communications Technologies, vol 251. Springer, Cham. https://doi.org/10.1007/978-3-031-87781-0_32

Gkonis, P., Giannopoulos, A., Trakadas, P., Masip-Bruin, X., & D'Andria, F. (2023). *A Survey on IoT-Edge-Cloud Continuum Systems: Status, Challenges, Use Cases, and Open Issues*. Future Internet, 15(12), 383. <https://doi.org/10.3390/fi15120383>

Google Cloud. (n.d). *Optimize Pod Autoscaling Based on Metrics*. Retrieved from

<https://cloud.google.com/kubernetes-engine/docs/tutorials/autoscaling-metrics>

Guruge, P. B., and Priyadarshana, Y. H.P. P. (2025, February 19th). Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and Long Short-Term Memory. Front. Comput. Sci., Sec. Software, Vol7 – 2025.

<https://doi.org/10.3389/fcomp.2025.1509165>

Guruvareddiar, S., Jagannathan, I., K., (2024). *Autoscaling Kubernetes workloads with KEDA using Amazon Managed Service for Prometheus metrics*. AWS. Retrieved from

<https://aws.amazon.com/blogs/mt/autoscaling-kubernetes-workloads-with-keda-using-amazon-managed-service-for-prometheus-metrics/>

Hakme, A., Gupta, R., and Kalasapurkar, U., (2022, April 05th). *7 patterns for IoT data ingestion and visualization- How to decide what works best for your use case*. Retrieved from <https://aws.amazon.com/blogs/iot/7-patterns-for-iot-data-ingestion-and-visualization-how-to-decide-what-works-best-for-your-use-case/>

Halleb, L., (2025, May 5th). *A Practical Example of The Custom Metrics Autoscaler*. Red Hat Developer. Retrieved from <https://developers.redhat.com/articles/2025/05/05/practical-example-custom-metrics-autoscaler>

Hendawi, A., Gupta, J., Liu, J. et al. Benchmarking large-scale data management for Internet of Things. *J Supercomput* **75**, 8207–8230 (2019). <https://doi.org/10.1007/s11227-019-02984-6>

Influxdata. *InfluxDB vs TimescaleDB – A Detail Comparison Compare influxDB and TimescaleDB for time series and OLAP workloads*. Retrieved from <https://www.influxdata.com/comparison/influxdb-vs-timescaledb/>

Jaeger (n.d.). *Architecture*. Retrieved from <https://www.jaegertracing.io/docs/1.23/architecture/>

KEDA (n.d.). *Kubernetes Event-driven Autoscaling*. Retrieved from <https://keda.sh/>

Krekovic, D., Krivic, P., Zarko, I. P., Le, P. D., (2024). *Reducing Communication Overhead in the IoT-Edge-Cloud Continuum: A Survey on Protocols and Data Reduction Strategies*. Retrieved from <https://arxiv.org/html/2404.19492v2>

Kubernetes. (2025, October 03). *Horizontal Pod Autoscaling*. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

Lee, F., (2023). *Kubernetes: HorizontalPodAutoscaler evaluation based on Prometheus metric*. Fabian Lee Software Engineer. Retrieved from <https://fabianlee.org/2023/09/09/kubernetes-horizontalpodautoscaler-evaluation-based-on-prometheus-metric/>

Mahgoub, A. (2025). *From monolith to microservices: building scalable applications with Kubernetes, CI/CD, and chaos engineering*. Retrieved from https://www.theseus.fi/bitstream/handle/10024/887394/Mahgoub_Ahmed.pdf

Majib, Y., Alosaimi, M., Asaturyan, A., and Perera, C., (2023, October 17). Dataset for Cyber-Physical Anomaly Detection in Smart Homes. *Front. Internet Things, Sec. Security, Privacy and Authentication*, Vol2 – 2023. <https://doi.org/10.3389/friot.2023.1275080>

Mishra, B., Mishra, B., & Kertesz, A. (2021). Stress-Testing MQTT Brokers: A Comparative Analysis of Performance Measurements. *Energies*, 14(18), 5817. <https://doi.org/10.3390/en14185817>

MongoDB (n.d.). *Bulk Write Operations*. Retrieved from <https://www.mongodb.com/docs/v8.0/core/bulk-write-operations/>

MongoDB (n.d.). *Modify a Cluster*. Retrieved from <https://www.mongodb.com/docs/atlas/scale-cluster/>

MongoDB. *Best Practices for Time Series Collections*. Retrieved from <https://www.mongodb.com/docs/manual/core/timeseries/timeseries-best-practices/>

MongoDB. *Shard a Time Series Collection*. Retrieved from <https://www.mongodb.com/docs/manual/core/timeseries/timeseries-shard-collection/>

MongoDB. *Time Series*. Retrieved from <https://www.mongodb.com/docs/manual/core/timeseries-collections/>

Nodejs. *Don't Block the Event Loop (or the Worker Pool)*. <https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop>

Nunes, J. P. K. S., Nejati, S., Sabetzadeh, M., and Nakagawa, E. Y., (2024, February 2nd). *Self-adaptive, Requirements-driven Autoscaling of Microservices*. Retrieved from <https://arxiv.org/pdf/2403.08798>

OASIS (2019, March 07). MQTT Version 5.0. Retrieved from <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

Owotogbe, J., Kumara, I., Heuvel, W-J. v. d., Tamburri, D. A., (2024). *Chaos Engineering: A Multi-Vocal Literature Review*. Retrieved from <https://arxiv.org/html/2412.01416v1>

Pilyai, X., Irsyad, R.N., Zaini, I.N., Negara, R.M., Hertiana, S.N., Tulloh, R. (2024). Implementation and Benchmarking of Kubernetes Horizontal Pod Autoscaling Method to Event-Driven Messaging System. In: Barolli, L. (eds) *Advances on Broad-Band and Wireless Computing, Communication and Applications. BWCCA 2023. Lecture Notes on Data Engineering and Communications Technologies*, vol 186. Springer, Cham. https://doi.org/10.1007/978-3-031-46784-4_5

Rodrigues, H., Rito Silva, A., Avritzer, A. (2024). *Performance Comparison of Monolith and Microservice Architectures*. In: Tekinerdoğan, B., Spalazzese, R., Sözer, H., Bonfanti, S., Weyns, D. (eds) *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium. ECSA 2023. Lecture Notes in Computer Science*, vol 14590. Springer, Cham. https://doi.org/10.1007/978-3-031-66326-0_12

Santos, J., Reppas, E., Wauters, T., Volckaert, B., and Turck, F. D., (February 2025). *Gwydion: Efficient auto-scaling for complex containerized applications in Kubernetes*

through Reinforcement Learning. Journal of Network and Computer Applications, Vol 234 – Feb 2025, 104067. <https://doi.org/10.1016/j.jnca.2024.104067>

Siddiqui, H., Khendek, F., and Toeroe, M., (2023). *Microservices based architectures for IoT systems – State-of-the-art review*. Internet of Things, vol 23. <https://doi.org/10.1016/j.iot.2023.100854>

Sinha, S., (2024). *State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally*. IOT ANALYTICS. Retrieved from <https://iot-analytics.com/number-connected-iot-devices>

Sivko, N., (2022). *How quickly can the Kubernetes HPA scale your app?*. Coroot. Retrieved from <https://coroot.com/blog/engineering/how-quickly-can-the-kubernetes-hpa-scale-your-app/>

Tapia, F., Mora, M. Á., Fuertes, W., Aules, H., Flores, E., & Toulkeridis, T. (2020). *From Monolithic Systems to Microservices: A Comparative Study of Performance*. Applied Sciences, 10(17), 5797. <https://doi.org/10.3390/app10175797>

X. Appendix

GitHub Repository (contain of the related codes & documents):

https://github.com/HaydenDuong/SIT314_729---Software-Architecture-and-Scalability-for-IOT/tree/main/Tasks/6.4HD

Video Demo: <https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=aca44efe-32e2-49d8-8256-b373005163e2>